

DAY 2

Generative AI & Prompt Engineering for Insurance Automation

Practical Training Content for UiPath Developers

Insurance Data	GenAI Prompts	UiPath JSON
----------------	---------------	-------------

Program Element	Details
Duration	4 Hours / Half Day
Audience	Beginner to Intermediate UiPath Developers, RPA Developers and Automation Engineers
Domain	Insurance claims, policy servicing, customer communication and underwriting support
Outcome	Create a reusable prompt library for insurance automation scenarios

1. Learning Objectives

By the end of Day 2, participants will be able to:

- Understand LLM fundamentals from an automation team perspective.
- Write practical prompts for insurance claim, policy and communication scenarios.
- Use system, user and developer-style instruction patterns effectively.
- Design structured JSON output that can be consumed by UiPath workflows.
- Create reusable prompt templates for enterprise insurance automation.
- Apply prompt improvement techniques such as role, context, rules and output format.

2. Recommended 4-Hour Session Flow

Time	Session Focus
30 min	LLM fundamentals for automation teams
40 min	Prompt engineering patterns for enterprise use cases
35 min	System, user and developer-style instruction design
45 min	Claim summary and policy summary hands-on labs
40 min	Customer email and risk indicator extraction labs
35 min	Structured JSON output for UiPath consumption
35 min	Prompt library assignment and trainer review

3. Big Picture: Why Generative AI for Insurance Automation?

Traditional RPA is excellent when data is structured and rules are clear. Insurance operations, however, often include emails, claim notes, scanned forms, customer messages, policy wording and underwriting remarks. Generative AI helps automation teams handle this unstructured information more intelligently.

Traditional Automation	Generative AI Automation
Reads fixed fields	Understands unstructured notes
Follows predefined rules	Generates summaries and explanations
Needs exact templates	Handles language variations
Moves data between systems	Creates structured outputs from messy inputs
Best for repetitive tasks	Best for language-heavy decisions and support tasks

4. Concept: LLM Fundamentals for Automation Teams

A Large Language Model, or LLM, is an AI model that can understand, summarize, classify and generate text. For automation teams, the LLM should be treated as an intelligent language component inside a larger automation workflow, not as a replacement for business rules, policy systems or human approval.

- Input: prompt, claim note, policy text, customer email or underwriting remarks.
- Processing: the model interprets context and generates a response.
- Output: summary, email, risk indicators, explanation or JSON.
- Integration: Python or UiPath can pass the output to downstream systems.

Example: LLM Use in a Claims Workflow

1. UiPath collects raw claim notes from email or claim system.
2. Python sends the claim note to the LLM with a clear prompt.
3. LLM returns a summary, missing document list and next action.
4. UiPath reads JSON output and routes the claim to the right queue.
5. Human reviewer validates sensitive or high-risk cases.

5. Sample Insurance Data for Hands-on Practice

The following sample records can be used across all Day 2 labs.

```
raw_claim_notes = """
Customer Rajesh Sharma submitted a health insurance claim on 12
June 2026.
He was admitted to City Care Hospital for dengue fever treatment.
Total bill amount is INR 85,000. Policy number is POL1001.
Documents include hospital bill, discharge summary and doctor
prescription.
The claim is currently pending document verification.
"""

policy_coverage_text = """
Health Secure Plus policy covers hospitalization expenses up to
INR 5,00,000.
Room rent is covered up to INR 5,000 per day. Pre-hospitalization
expenses are
covered for 30 days and post-hospitalization expenses are covered
for 60 days.
Cosmetic treatment and experimental treatment are excluded.
"""
```

6. Concept: Prompt Engineering Pattern

Prompt engineering means giving clear instructions to the LLM so that the output is useful, consistent and automation-ready. A good enterprise prompt should include role, context, task, rules and output format.

Prompt Element	Purpose
Role	Defines who the AI should act as.
Context	Provides business background and process meaning.
Task	Clearly states what the AI must do.
Rules	Defines guardrails, restrictions and compliance boundaries.
Output Format	Controls whether the answer should be bullets, email, table or JSON.

Weak Prompt vs Better Prompt

Weak Prompt	Better Prompt
Summarize this claim.	You are an insurance claims assistant. Summarize the claim note in 5 bullet points. Include customer name, policy number, claim amount, documents and current status.

Hands-on Lab 1: Generate Claim Summary from Raw Claim Notes

Objective: Create a concise operational claim summary for a claims processing team.

Steps:

1. Copy the sample raw claim notes into a Python string.
2. Create a prompt with role, task and output format.
3. Ask the model to summarize the claim in bullets.
4. Review whether customer name, claim amount, policy number and

documents are included.

5. Improve the prompt if any important detail is missing.

Expected Output: A clear claim summary containing customer name, policy number, claim type, amount, documents and status.

Mini Assignment: Create two versions: one for claims manager and one for customer support executive.

```
prompt = f"""
```

```
You are an insurance claims assistant.
```

```
Task:
```

```
Create a concise claim summary from the raw claim notes.
```

```
Include:
```

- Customer name
- Policy number
- Claim type
- Claim amount
- Documents submitted
- Current status

```
Raw Claim Notes:
```

```
{raw_claim_notes}
```

```
"""
```

7. Concept: System, User and Developer-Style Instructions

Instruction layering helps enterprise teams control model behavior. This is useful when the same AI capability is reused across multiple UiPath workflows.

Instruction Type	Meaning and Example
System Instruction	High-level model behavior. Example: You are a professional insurance automation assistant.
Developer-Style Instruction	Application rules. Example: Always return missing documents if visible.
User Instruction	Task-specific request. Example: Summarize this claim note.

For UiPath developers, this is similar to separating process configuration, reusable business rules and transaction-specific input.

```
system_instruction = """
You are a professional insurance automation assistant.
Use clear, neutral and compliance-friendly language.
Do not approve or reject claims without policy evidence.
"""

developer_instruction = """
Always structure the answer for downstream automation.
If documents are missing, mention them clearly.
Avoid emotional or promotional language.
"""

user_instruction = f"""
```

Create a claim summary from the following notes:

```
{raw_claim_notes}  
"""
```

Hands-on Lab 2: Create Policy Coverage Explanation

Objective: Convert policy wording into simple customer-friendly explanation without changing policy meaning.

Steps:

1. Read the policy coverage text.
2. Write a role-based prompt for customer support.
3. Ask for covered items, exclusions, limits and caution note.
4. Check whether the response is simple, accurate and non-legalistic.
5. Revise the prompt to reduce ambiguity.

Expected Output: A customer-friendly explanation with covered benefits, exclusions, limits and caution note.

Mini Assignment: Create one version for customer email and one version for internal support team.

```
policy_prompt = f"""
```

You are an insurance customer support assistant.

Explain the following policy coverage in simple language.

Include:

1. What is covered
2. What is not covered
3. Coverage limits
4. Simple caution note

Policy Text:


```
{policy_coverage_text}
"""
```

8. Concept: Professional Customer Communication

Insurance communication must be polite, accurate and careful. The AI should not promise approval unless the claim has already been formally approved by the claim system or authorized reviewer.

- Use professional and empathetic tone.
- Avoid final approval or rejection language unless confirmed.
- Mention current status clearly.
- Ask for missing documents in a simple way.
- Keep the message short and action-oriented.

Avoid	Prefer
Your claim will be approved soon.	Your claim is currently under verification.
We guarantee payment.	We will update you after document review is completed.
You failed to submit documents.	Please share the pending documents to continue processing.

Hands-on Lab 3: Generate Professional Customer Email

Objective: Draft a customer email response for a claim status query.

Steps:

1. Create a customer query string.
2. Create a prompt with tone and compliance rules.
3. Ask the model to draft a short professional email.
4. Check whether the email avoids false commitments.
5. Create a second version as SMS or WhatsApp-style update.

Expected Output: A polite email that gives status, asks for pending documents if

required and avoids approval promises.

Mini Assignment: Create formal, short and internal-note versions of the same communication.

```
customer_query = """
```

```
Dear Team, I have submitted my hospital claim but I have not  
received any update.
```

```
Please let me know whether my claim is approved and if any  
documents are pending.
```

```
Regards,
```

```
Rajesh Sharma
```

```
"""
```

```
email_prompt = f"""
```

```
You are a professional insurance customer support executive.
```

```
Task:
```

```
Draft a polite customer email response.
```

```
Rules:
```

- Do not confirm approval.
- Mention current status.
- Ask for pending documents if needed.
- Use professional tone.
- Keep it short.

```
Customer Email:
```

```
{customer_query}
```

```
"""
```

9. Concept: Extracting Risk Indicators

Risk indicators are signals that require additional verification. They do not automatically mean fraud. The prompt must use neutral language and should never accuse the customer.

- Recently purchased policy before claim event.
- Missing police report or supporting document.
- High claim amount compared with incident description.
- Blurry or incomplete photos.
- Repeated claims or inconsistent information.
- Delayed reporting without clear explanation.

Hands-on Lab 4: Extract Risk Indicators from Claim Description

Objective: Identify risk signals and recommend next verification steps using neutral language.

Steps:

1. Read the claim description carefully.
2. Write a prompt that asks for risk indicators, missing documents and next action.
3. Ask the model to classify risk as Low, Medium or High.
4. Review whether the output is neutral and not accusatory.
5. Convert the risk output into structured bullets.

Expected Output: A neutral risk review containing indicators, missing documents, risk level and next verification action.

Mini Assignment: Add customer questions that can help clarify the claim.

```
claim_description = ""  
Customer reported vehicle damage after a minor road accident.  
The accident happened at midnight. The customer submitted repair  
estimate of INR 2,40,000.  
Policy started only 5 days before the accident. Police report is  
not attached.
```

```
Photos are blurry and garage estimate appears unusually high for minor damage.
```

```
"""
```

```
risk_prompt = f"""
```

```
You are an insurance claim risk review assistant.
```

```
Task:
```

```
Extract risk indicators from the claim description.
```

```
Rules:
```

- Do not accuse the customer.
- Use neutral language.
- Assign risk level: Low, Medium or High.
- Suggest next verification step.

```
Claim Description:
```

```
{claim_description}
```

```
"""
```

10. Concept: Underwriting Notes Generation

Underwriting notes help reviewers understand important facts, concerns and decision-support points. The AI should support underwriters by organizing information, not by making final binding decisions.

- Summarize applicant or claim background.
- Identify important risk factors.
- Highlight missing information.
- Suggest questions for further review.
- Prepare structured notes for human underwriter.

```
underwriting_prompt = f"""
```

```
You are an underwriting support assistant.
```

```
Prepare underwriting notes from the claim description.
```

Include:

- Key facts
- Risk observations
- Missing information
- Questions for reviewer
- Recommended next review step

Claim Description:

```
{claim_description}
"""
```

11. Concept: Structured Output and JSON Response Design

UiPath works best with structured and predictable data. Therefore, many GenAI outputs should be generated as JSON instead of free-form paragraphs.

- JSON is easy for UiPath to parse.
- JSON supports downstream decision rules.
- JSON can be stored in databases or audit logs.
- JSON reduces manual copy-paste and formatting errors.
- JSON must be validated before production use.

Recommended JSON Fields for Claim Automation

Field	Purpose
claim_id	Unique claim reference
policy_number	Policy linked to the claim
customer_name	Customer or insured person
claim_type	Health, vehicle, life, property or travel
claim_amount	Requested amount
summary	Short claim summary
risk_indicators	List of risk signals
missing_documents	List of pending documents
recommended_action	Next step for UiPath or human reviewer

Hands-on Lab 5: Produce JSON Output for UiPath Consumption

Objective: Generate valid JSON from claim notes so that UiPath can parse and route the transaction.

Steps:

1. Create a prompt that asks for JSON only.
2. Define all required JSON fields clearly.
3. Ask the model to use null if information is missing.
4. Validate whether the output is parseable JSON.
5. Use the JSON fields for UiPath decision routing.

Expected Output: Valid JSON containing claim summary, risk indicators, missing documents and recommended action.

Mini Assignment: Create a JSON schema for health claim and vehicle claim separately.

```
json_prompt = f"""
You are an insurance automation assistant.
Return valid JSON only. Do not add explanation outside JSON.
```

Required JSON fields:

- claim_id
- policy_number
- customer_name
- claim_type
- claim_amount
- summary
- risk_indicators
- missing_documents
- recommended_action

Raw Claim Notes:

```
{raw_claim_notes}
"""
```

```
{
  "claim_id": "CLM9001",
  "policy_number": "POL1001",
  "customer_name": "Rajesh Sharma",
  "claim_type": "Health",
  "claim_amount": 85000,
  "summary": "Health claim for dengue treatment at City Care Hospital.",
  "risk_indicators": [],
  "missing_documents": [],
  "recommended_action": "Proceed with document verification"
}
```

12. Concept: JSON Validation Before UiPath

LLM output should not be blindly trusted in enterprise automation. Python or UiPath should validate the output before using it in the workflow.

- Check whether output is valid JSON.
- Check whether required fields are present.
- Check whether numeric fields are numeric.
- Check whether list fields are actually lists.
- Check whether action values are from approved options.

```
required_fields = [
    "claim_id", "policy_number", "customer_name", "claim_type",
    "claim_amount", "summary", "risk_indicators",
    "missing_documents", "recommended_action"
]

def validate_json_output(output):
    errors = []
    for field in required_fields:
        if field not in output:
            errors.append(f"Missing field: {field}")
```

```

if "claim_amount" in output and not
instance(output["claim_amount"], (int, float)):
    errors.append("claim_amount must be numeric")
return errors

```

13. Prompt Library for Insurance Automation

A prompt library is a reusable collection of approved prompts. It improves consistency, governance and productivity across automation teams.

Prompt Template	Business Use
Claim Summary Prompt	Summarize claim notes for operations team
Policy Explanation Prompt	Explain policy terms to customer support
Customer Email Prompt	Draft safe and professional customer response
Risk Indicator Prompt	Identify signals requiring verification
Underwriting Notes Prompt	Prepare structured reviewer notes
UiPath JSON Prompt	Generate structured output for workflow routing

Prompt Template Format

Section	Description
Prompt Name	Clear reusable name
Purpose	Where it will be used
Role	Who AI should act as
Context	Business background
Task	Specific action
Rules	Guardrails and restrictions
Output Format	Bullets, email, table or JSON
Sample Input	Example insurance text
Expected Output	Ideal response format

14. Final Assignment: Create Prompt Library

Assignment Outcome: Participants will create a prompt library for insurance automation scenarios. This prompt library should be reusable in Python, UiPath, LangChain or future Agentic AI workflows.

Required Prompt Templates

1. Claim Summary Prompt
2. Policy Coverage Explanation Prompt
3. Professional Customer Email Prompt
4. Risk Indicator Extraction Prompt
5. Underwriting Notes Prompt
6. JSON Output Prompt for UiPath Consumption

Assignment Steps

1. Select one insurance scenario such as health claim, vehicle claim or policy servicing.
2. Create sample input text for the selected scenario.
3. Design prompts using role, context, task, rules and output format.
4. Run prompts and review whether output is complete and safe.
5. Improve prompts based on missing information or unclear response.
6. Prepare the final prompt library in table or JSON format.

Evaluation Criteria

Criteria	Weightage
Prompt clarity and completeness	25%
Insurance domain relevance	20%
Output format quality	20%
Guardrails and compliance-friendly language	20%
Reusability for UiPath automation	15%

15. Small Practice Exercises

- Rewrite a weak prompt into an enterprise-grade prompt.

- Generate the same claim summary for three audiences: customer, claims manager and audit team.
- Create a JSON-only prompt for UiPath queue processing.
- Create a customer email that avoids approval commitment.
- Extract missing documents from claim notes.
- Create underwriting notes from a vehicle claim description.

16. Trainer Discussion Questions

1. Which insurance processes are most suitable for Generative AI?
2. Where should human approval remain mandatory?
3. Why is JSON important for UiPath developers?
4. What can go wrong when prompts are vague?
5. How can prompt libraries improve governance?
6. How can Day 2 prompts connect with Day 1 Python automation?
7. Where can LangGraph agents be added in future sessions?

17. Day 2 Summary

Day 2 helps UiPath developers move from basic AI awareness to practical prompt engineering for insurance automation. Participants learn how to convert unstructured insurance text into summaries, emails, risk indicators and structured JSON output.

- LLMs are useful for language-heavy insurance automation tasks.
- Good prompts require role, context, task, rules and output format.
- Instruction layering improves control and reusability.
- Customer communication must be professional and non-committal.
- JSON output makes GenAI useful for UiPath workflows.
- Prompt libraries provide repeatability, governance and faster delivery.

Connection to Future Sessions

- Day 3: LangChain assistants and tool-based AI workflows.
- Day 4: Agentic AI and LangGraph for multi-step claim workflows.
- Day 5 onward: MCP, UiPath integration and enterprise capstone.